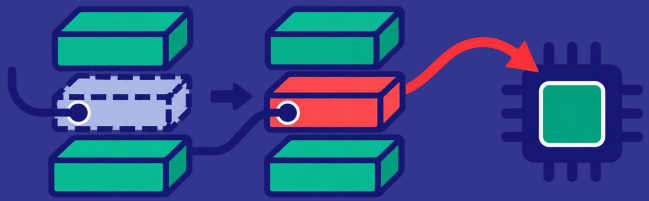
بسم الله الرحمن الرحیم

Data and Network Security

SESSION 05

Advanced Memory Vulnerabilities



Solmaz Salimi
Sharif University of Technology

s.salimi@sharif.edu

Last lecture and today's lecture

Session 4	Stack frames and corrupted return addresses
Part 1	Integer conversion, overflow, and buffer sizes
Part 2	Format strings and GOT indirection
Part 3	Heap objects, callbacks, and virtual dispatch
Part 4	Use-after-free and object lifetime
Checkpoint	Trace the bug, the primitive, and the consequence








Beyond the saved return address

Several bug classes can reach the same control boundary.

Integer error	**A wrong length leads to an invalid memory access**
Format string	**Input controls how a function interprets arguments**
Heap overflow	**A write leaves the allocated object**
Use-after-free	**An access occurs after the object lifetime ends**

Spatial safety concerns bounds. Temporal safety concerns lifetime.

Part 1: Integer errors and buffer sizes



Integer semantics determine the result

UNSIGNED

Arithmetic wraps modulo 2^w.
For uint32_t:
UINT32_MAX + 1u becomes 0.

SIGNED

An out-of-range signed arithmetic result has undefined behavior in C.
A wraparound result is not guaranteed.

Conversions and narrowing also change values. Validate the value in the type used by the operation.

Here w is the width of the arithmetic type after integer promotions.

A negative length becomes a huge copy

```
char buf[16];
int len = read_len();
if (len > 16) return;
memcpy(buf, src, len);
```

len = -1	**len > 16 is false**
check	**lower bound is missing**
conversion	**size_t becomes SIZE_MAX**

Reject negative lengths before converting to size_t.

Assume src is valid for the intended input length.

An overflowing sum can pass a bounds check

```
char tmp[256];
uint32_t a, b;
if (a + b > 256) return;
memcpy(tmp, s1, a);
memcpy(tmp + a, s2, b);
```

a = 128 — first copy fits

b = 2^32-128 — second copy is enormous

sum = 0 — the guard passes

Check a <= capacity, then b <= capacity - a.

Assume 32-bit uint32_t arithmetic and valid source buffers.

An allocation can be smaller than its logical size

```
size_t bytes = count * 8;
char *p = malloc(bytes);
if (!p) return;
for (size_t i=0; i<count; ++i)
    copy_item(p + 8*i, i);
```

count	2^61 + 1 items
bytes	8 after wraparound
writes	still follow count

On 64-bit size_t, the product wraps to 8 bytes.

copy_item writes exactly 8 bytes. This is a simplified fragment.

Checks before arithmetic and conversion

```c
if (len < 0) return;
size_t n = (size_t)len;
if (n > capacity) return;

if (count > SIZE_MAX / 8)
    return;
size_t bytes = count * 8;
```

Range	Validate lower and upper bounds
Type	Use the validated size_t value
Product	Check division before multiplication
Policy	Also bound legitimate resource use

A mathematically valid size may still exhaust memory. Enforce an application-level size limit.

One extra byte can corrupt adjacent state

```
char buf[16];
size_t n = 16;
if (n > sizeof buf) return;
memcpy(buf, src, n);
buf[n] = '\0';
```

OFF-BY-ONE

Where does the terminator land?

buf[16] is outside the array. Valid indices are 0 through 15.

- A string needs space for the terminator
- Require n < sizeof buf before this sequence
- Impact depends on the neighboring field

Part 2: Format strings and GOT indirection



A format string is an instruction to printf

VULNERABLE

```
char input[] = "%p";
printf(input);
```

The input chooses conversion directives and expected argument types.

No pointer argument: undefined behavior.

FIXED FORMAT

```
char input[] = "%p";
printf("%s", input);
```

The program chooses the format.
The input is one string argument.

Output: %p (printed literally)

A bounded input can still be a format-string vulnerability. Input length and format control are separate properties.

The same rule applies to fprintf, snprintf, syslog, and other format-based APIs.

Format directives can read or write



%p	**Print a supplied pointer value**
%s	Read a string through a supplied pointer
%n	Write the character count through int *
%%	Print a literal percent sign, consuming no argument

int n; printf("hello%n", &n); writes 5 into n. It prints no characters for %n.

%n writes a count into memory

EXAMPLE

```
int n = -1;
printf("hello%n world\n", &n);
printf("n = %d\n", n);
```

Output:
hello world
n = 5

WHAT %n DOES

At %n: 5 characters printed.

&n supplies a writable int *.
%n stores 5 in n.
%n itself prints nothing.

The remaining text prints next.

The count is taken when %n is reached, not at the end of the call.

A controlled format can cause a memory write when a usable destination pointer is consumed.

Controlled formats create memory-access primitives

```
char msg[128];
if (!fgets(msg, sizeof msg,stdin))
    return;
printf(msg);
```

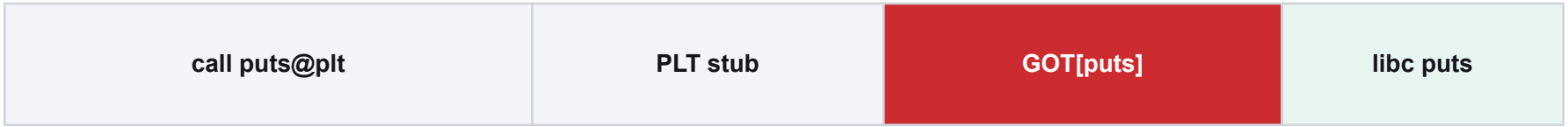
format	attacker controls directives
argument	missing arguments: undefined behavior
effect	leak, crash, or write

Exploitation requires usable pointer values and a controllable argument interpretation.

Use printf("%s", msg) or fputs(msg, stdout).

Dynamic calls use indirection through the GOT

```
ELF dynamic linking: call site → PLT stub → GOT slot → function
```



caller invokes a stub

loads a GOT slot

resolved function address

execution continues

The dynamic loader resolves imported symbols. GOT overwriting requires a writable slot and a suitable memory-write primitive.

When are imported function addresses resolved?

LAZY BINDING

First call:
PLT reaches the resolver.
The symbol is looked up.
The GOT slot is updated.

Later calls reuse that address.

EAGER BINDING

Before normal execution:
Imported functions are resolved.
Their GOT slots are initialized.

Calls use resolved addresses.

Link option: -Wl,-z,now

PLT contains executable stubs. GOT contains address entries.

This is the conventional PLT/GOT path. Details depend on the architecture and build options.

A writable GOT slot is a conditional control target



?	**Primitive**	Can the bug write the desired address and value?
?	**Location**	Can the attacker locate the target GOT slot?
?	**Writable**	Do page protections permit the write?
?	**Trigger**	Will the program later call that imported function?
✓	**Defense**	Full RELRO closes this particular write target

RELRO protects relocation state. It does not repair the memory-safety bug that supplied the write.

Partial and full RELRO protect different targets

PARTIAL RELRO

```
-Wl,-z,relro
```

Some relocation data becomes read-only after initialization.

Lazy binding keeps function GOT slots writable in the typical PLT/GOT configuration.

FULL RELRO

```
-Wl,-z,relro,-z,now
```

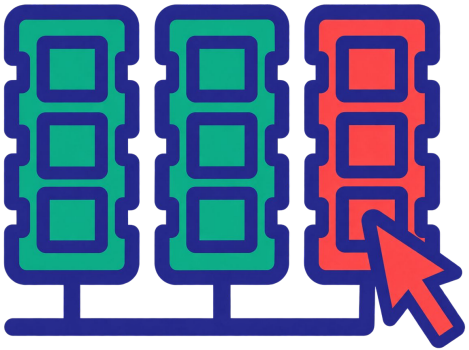
Function symbols are resolved before normal execution.

Protected GOT slots become read-only, blocking overwrites.

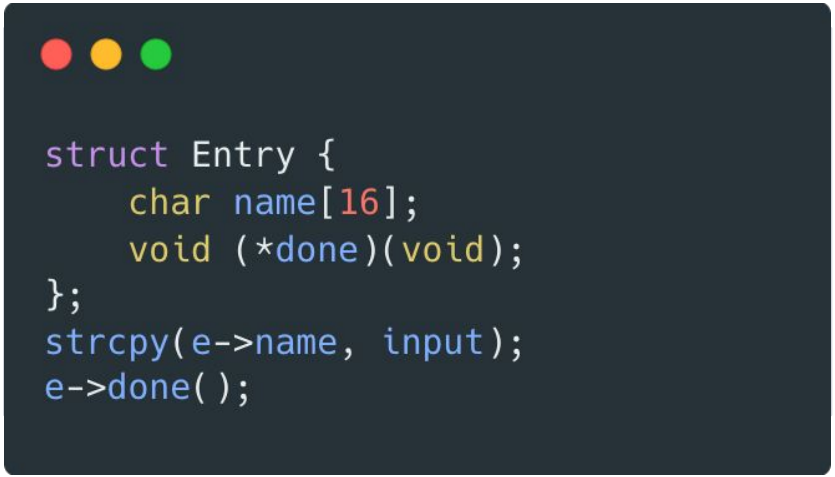
RELRO means Relocation Read-Only. Full RELRO blocks this GOT overwrite target.

The format-string bug remains. Repair it with a fixed format, such as printf("%s", input).

Part 3: Heap objects and control data



A heap overflow can overwrite a callback



```
struct Entry {
    char name[16];
    void (*done)(void);
};
strcpy(e->name, input);
e->done();
```

name[16]	fixed capacity
done	function pointer follows
indirect call	uses corrupted address

The same allocation can hold both input data and a control pointer.

Assume e points to a live Entry with done initialized.

A same-object overflow changes the callback



LIVE OBJECT	AFTER OVERFLOW
Entry.name[16]	attacker bytes
Entry.done	Entry.done overwritten
valid function address	attacker-chosen target

At e->done(), the program uses the stored function address. No saved return address needs to change.

Virtual calls select a method from the actual object



CLASS DEFINITIONS

```
struct Animal {
  virtual void speak() = 0;
  virtual ~Animal() = default;
};
struct Cat : Animal {
  void speak() override { puts("Meow"); }
};
struct Dog : Animal {
  void speak() override { puts("Woof"); }
};
```

SAME CALL, TWO RESULTS

```
void make_sound(Animal *a) {
  a->speak();
}

Animal *a = new Cat;
Animal *b = new Dog;
make_sound(a);  // Meow
make_sound(b);  // Woof
delete a; delete b;
```

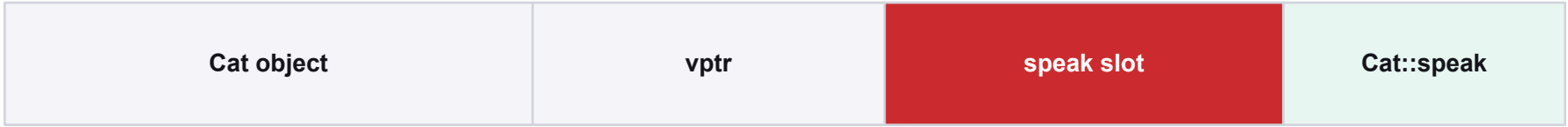
Animal* is the parameter type.
Cat or Dog is the actual object type.

virtual selects the override for the actual object at runtime.

Like the earlier callback, virtual dispatch uses stored data to select code.

Virtual dispatch adds a second level of indirection

```
a->speak(): object → vptr → speak slot → Cat::speak
```



seen through Animal *

points to a virtual table

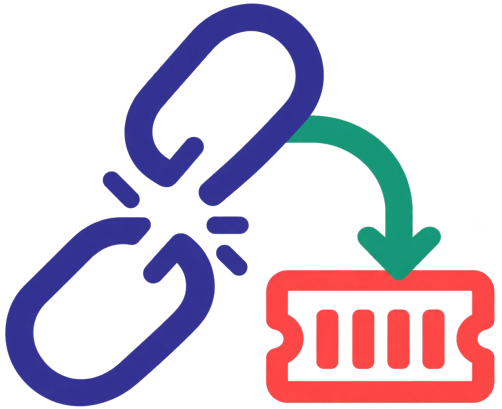
selects Cat::speak

prints Meow

Callback: object → function. Virtual call: object → table → function.

An overflow may corrupt a neighboring object's vptr. Adjacency and offsets depend on the allocator and ABI.

Part 4: Use-after-free and object lifetime



make_entry allocates and initializes one Entry



OBJECT AND CALLBACK

```
#include <stdlib.h>
#include <stdio.h>

typedef struct {
  char name[16];
  void (*done)(void);
} Entry;

void finished(void) {
  puts("Done");
}
```

FACTORY FUNCTION

```
Entry *make_entry(void) {
  Entry *e = malloc(sizeof *e);
  if (!e) return NULL;
  e->name[0] = '\0';
  e->done = finished;
  return e;
}

Entry *p = make_entry();
if (!p) return;
```

malloc creates the object; done stores the finished function address.

The caller owns the returned object and must free it after its last use.

Freeing an object leaves aliases behind

```
Entry *p = make_entry();
Entry *alias = p;
free(p);
p = NULL;
/* ... other work ... */
alias->done();
```

free(p) ends object lifetime

p = NULL changes one variable

alias still refers to old object

Nulling one pointer cannot invalidate every alias. The final access is undefined behavior.

Assume make_entry returns a malloc-allocated, initialized object.

The same address can later hold a different object



1 Allocate

p and alias refer to a live Entry object

valid lifetime

2 Free + reuse

Entry lifetime ends new data may occupy its chunk

stale alias

3 Stale access

alias interprets the new bytes as Entry fields

type confusion

A UAF attack needs a stale access. Reliable control also depends on reuse, chosen bytes, and the later operation.

Case study: a callback invalidates pending work

Callback: a function invoked for an event. Here it runs on the same thread.

RESET	Reset restores form fields to their initial values.
CALLBACK	A registered event handler runs during reset. It removes controls, ending their lifetime.
REUSE	New allocations may reuse the freed memory. UAF does not require this reuse.
RESUME	The handler returns; reset continues. It accesses a freed control through a stale pointer.

CVE-2014-0282: the critical mistake is continuing after a callback changed object lifetime.

Repair bounds, format control, and ownership

✓	**Arithmetic**	Check representable sizes before arithmetic
✓	**Bounds**	Track object capacity, including terminators
✓	**Formats**	Use fixed format strings with typed arguments
✓	**Lifetime**	Define one owner and make alias validity explicit
✓	**Evidence**	ASan / UBSan expose many bugs during testing

Double-free releases one allocation twice. Allocator checks may abort; ownership must prevent the second release.

Checkpoint: find the first broken invariant

An asynchronous handler

```
Entry *p = make_entry();
queue_callback(p);
free(p);
/* callback runs later */
```

1 Which event makes the queued pointer stale?

2 Would p = NULL repair the queued callback?

3 Which owner must keep the object alive?

free(p) ends the lifetime. Nulling p changes no queued alias. The queue must own valid state or cancel the callback before release.

References

BERKELEY CS 161: Memory Safety Vulnerabilities, Ch. 3. Format strings, integer conversion, off-by-one, heap and lifetime errors.

STANFORD CS 155 Lecture 2: Control Hijacking Attacks. blexim, Basic Integer Overflows (Phrack 60, 2002).

UAF CASE Katy Winterborn, Exploiting CVE-2014-0282, NCC Group, 2015.

PRIOR TERM Prior-term control-hijacking slides and supplements. ELF / GNU ld documentation for GOT and RELRO.

Next: defenses against control hijacking

Session 6 asks which part of an attack each defense blocks.

NX / DEP	Restricts execution from writable memory
Canaries	Detects some stack corruption before return
ASLR	Makes useful addresses harder to predict
Code reuse	Return-to-libc and ROP use existing instructions

Today's review chain: input → bug → memory primitive → security consequence.